

Cross Validation

Hakan Gogtas

Cross Validation

Cross-validation is a model evaluation technique generally used to evaluate a machine learning algorithm's performance in making predictions on new datasets that it has not been trained on. In fact, it is not advisable to compare the predictive accuracy of a set of models using the same observations as used for model estimation. Therefore, to evaluate the predictive performance of the models, we must use an independent set of data.

In the cross-validation procedure, a dataset partitions a subset of data used to train the algorithm, and the remaining data is used for testing. Subdivision is usually randomly performed to ensure that the two parts have the same distribution. Because cross-validation does not use all of the data to build a model, it is a commonly used method to detect overfitting during training.

70-30 Split

In cross-validation, each round involves randomly partitioning the original dataset into a training set and a testing set. The training set is then used to train a regression algorithm, and the testing set is used to evaluate its performance. So, the model is fit on the training set, and the fitted model is used to predict the responses for the observations in the testing set. This process is repeated several times, and the average cross-validation error is used as a performance indicator. The model showing the lowest error on the test set is identified as the best.

Several cross-validation techniques are available. The following lists the most used:

k-fold: This technique subdivides data into k randomly chosen subsets (named folds) of roughly equal sizes. One subset is used to validate the model, while the remaining subsets are used for training. This process is repeated k times, such that each subset is used exactly once for validation.

Leave-one-out cross-validation (LOOCV): This technique subdivides data using the k -fold approach, where k is equal to the total number of observations in the data. This time, a single observation is used for the testing set, and the remaining observations make up the training set.

In addition, two more resampling methods are available:

Data splitting: This technique subdivides into exactly two subsets of specified ratios for training and validation.

Bootstrap resampling: This technique creates bootstrap samples by randomly collecting observations from the original dataset with replacements against which to evaluate the model. Typically, a large number of resampling iterations are performed.

To improve our practice with the cross-validation techniques, we look at an example. This time, we will use the Motor Trend Car Road Tests dataset (mtcars) contained in the datasets package. The data was extracted

from the 1974 Motor Trend US magazine and comprises fuel consumption and ten aspects of automobile design and performance for 32 automobiles (1973–74 models). The data consists of 32 observations on 11 variables, as follows:

Mpg: Miles/(US) gallon cyl: Number of cylinders disp: Displacement (cu.in.) hp: Gross horsepower drat: Rear axle ratio wt: Weight (1000 lbs) qsec: 1/4 mile time vs: V/S am: Transmission (0 = automatic, 1 = manual) gear: Number of forward gears carb: Number of carburetors

Let's start by loading the mtcars dataset through the following command which, as we anticipated, is contained in the dataset's library and saves it in a given frame:

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
data(mtcars)
```

```
str(mtcars)
```

```
## 'data.frame': 32 obs. of 11 variables:
## $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
## $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
## $ disp: num 160 160 108 258 360 ...
## $ hp : num 110 110 93 110 175 105 245 62 95 123 ...
## $ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
## $ wt : num 2.62 2.88 2.32 3.21 3.44 ...
## $ qsec: num 16.5 17 18.6 19.4 17 ...
## $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
## $ am : num 1 1 1 0 0 0 0 0 0 0 ...
## $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
## $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
summary(mtcars)
```

```
##      mpg          cyl          disp          hp
## Min.   :10.40   Min.   :4.000   Min.    : 71.1   Min.    : 52.0
## 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
## Median :19.20   Median :6.000   Median :196.3   Median :123.0
## Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
## 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
## Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0
##      drat          wt          qsec          vs
## Min.    :2.760   Min.    :1.513   Min.    :14.50   Min.    :0.0000
## 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
## Median :3.695   Median :3.325   Median :17.71   Median :0.0000
## Mean    :3.597   Mean    :3.217   Mean    :17.85   Mean    :0.4375
## 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
## Max.    :4.930   Max.    :5.424   Max.    :22.90   Max.    :1.0000
##      am          gear          carb
## Min.    :0.0000   Min.    :3.000   Min.    :1.000
## 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000
## Median :0.0000   Median :4.000   Median :2.000
## Mean    :0.4062   Mean    :3.688   Mean    :2.812
## 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
## Max.    :1.0000   Max.    :5.000   Max.    :8.000
```

```
R<-round(cor(mtcars),2)
R
```

```
##      mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
## mpg   1.00 -0.85 -0.85 -0.78  0.68 -0.87  0.42  0.66  0.60  0.48 -0.55
## cyl  -0.85  1.00  0.90  0.83 -0.70  0.78 -0.59 -0.81 -0.52 -0.49  0.53
## disp -0.85  0.90  1.00  0.79 -0.71  0.89 -0.43 -0.71 -0.59 -0.56  0.39
## hp   -0.78  0.83  0.79  1.00 -0.45  0.66 -0.71 -0.72 -0.24 -0.13  0.75
## drat  0.68 -0.70 -0.71 -0.45  1.00 -0.71  0.09  0.44  0.71  0.70 -0.09
## wt   -0.87  0.78  0.89  0.66 -0.71  1.00 -0.17 -0.55 -0.69 -0.58  0.43
## qsec  0.42 -0.59 -0.43 -0.71  0.09 -0.17  1.00  0.74 -0.23 -0.21 -0.66
## vs    0.66 -0.81 -0.71 -0.72  0.44 -0.55  0.74  1.00  0.17  0.21 -0.57
## am    0.60 -0.52 -0.59 -0.24  0.71 -0.69 -0.23  0.17  1.00  0.79  0.06
## gear  0.48 -0.49 -0.56 -0.13  0.70 -0.58 -0.21  0.21  0.79  1.00  0.27
## carb -0.55  0.53  0.39  0.75 -0.09  0.43 -0.66 -0.57  0.06  0.27  1.00
```

The correlation coefficient has to be between -1.0 and 1.0 (0 = no correlation; $-1/1$ =high negative/positive correlation). We notice that the mpg variable is largely correlated with the variables wt, cyl, and disp. We will confirm this in the subsequent analyzes.

The following example divides the dataset so that 70 percent is used to train a linear regression model and the remaining 30 percent is used to evaluate model performance. To perform data splitting, we can use the createDataPartition() function (contained in the caret package) that returns a list or matrix of row position integers corresponding to the training data. For bootstrap samples, simple random sampling is used. For other data splitting, the random sampling is done within the levels of y, when y is a factor, in an attempt to balance the class distributions within the splits. For numeric y, the sample is split into groups based on percentiles, and sampling is done within these subgroups. The general form of the function is shown in the following:

```
DataSplit<-createDataPartition(y = mtcars$mpg, p = 0.7, list = FALSE)
TrainData<-mtcars[DataSplit,]
TestData<-mtcars[-DataSplit,]
```

In these two lines, the data of the data frame named mtcars is subdivided into two new data frames, called Train Data and Test Data. Now we can build the model: to do this, we will use the train() function. This function can be used to estimate the coefficient values for various modeling functions, like regression, and others. This function sets up a grid of tuning parameters, and can also compute resampling based performance measures. Let's see how it works:

```
LmFit1<-train(mpg~., data = TrainData, method = "lm")
```

```
## Warning in predict.lm(modelFit, newdata): prediction from rank-deficient fit;
## attr(*, "non-estim") has doubtful cases
```

```
f<-lm(mpg~., data = TrainData)
```

A string specifying which regression model to adopt has been used (in our case, lm). Possible values are found using a names (getModelInfo()) command. To look at the model, let's see a brief summary using the summary() function:

```
summary(LmFit1)
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -3.1809 -1.6313 -0.2131 1.2084 4.3929
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -3.65173   33.55517  -0.109  0.9150
## cyl         -0.67353    1.35981  -0.495  0.6286
## disp         0.02036    0.02294   0.887  0.3910
## hp          -0.01290    0.02921  -0.442  0.6660
## drat         1.20367    2.15716   0.558  0.5863
## wt          -4.76497    2.68766  -1.773  0.0997 .
## qsec         1.70329    1.41419   1.204  0.2499
## vs          -2.77061    4.40870  -0.628  0.5406
## am           0.20653    2.99296   0.069  0.9460
## gear         1.90052    2.37613   0.800  0.4382
## carb        -0.02503    1.11468  -0.022  0.9824
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.908 on 13 degrees of freedom
## Multiple R-squared:  0.8591, Adjusted R-squared:  0.7507
## F-statistic: 7.927 on 10 and 13 DF, p-value: 0.0004631
```

```
summary(f)
```

```
##
## Call:
## lm(formula = mpg ~ ., data = TrainData)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.1809 -1.6313 -0.2131  1.2084  4.3929
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -3.65173   33.55517  -0.109  0.9150
## cyl         -0.67353    1.35981  -0.495  0.6286
## disp         0.02036    0.02294   0.887  0.3910
## hp          -0.01290    0.02921  -0.442  0.6660
## drat         1.20367    2.15716   0.558  0.5863
## wt          -4.76497    2.68766  -1.773  0.0997 .
## qsec         1.70329    1.41419   1.204  0.2499
## vs          -2.77061    4.40870  -0.628  0.5406
## am           0.20653    2.99296   0.069  0.9460
## gear         1.90052    2.37613   0.800  0.4382
## carb        -0.02503    1.11468  -0.022  0.9824
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.908 on 13 degrees of freedom
## Multiple R-squared:  0.8591, Adjusted R-squared:  0.7507
## F-statistic: 7.927 on 10 and 13 DF, p-value: 0.0004631
```

A linear regression model that uses the least squares approach to determine optimal parameters for the given data has been returned. Now, a test must be performed on the Test Data or other new data using parameter estimates obtained from the model building process. The following shows how the testing activity

is performed on the Test Data sample using the coefficients obtained from the model built on the Train Data sample. This step can be easily implemented with the help of the `predict()` function. This function is a generic function for predictions from the results of various model fitting functions. The function invokes particular methods, which depend on the class of the first argument:

```
PredictedTest<-predict(LmFit1,TestData)
```

At this point, we have to build data.frame with the values of the current and estimated mpg variable to compare them:

```
ModelTest1<-data.frame(obs = TestData$mpg, pred=PredictedTest)
```

To see model performance metrics on the TestData sample, you can use a `defaultSummary()` function that, given two numeric vectors of data, calculates the mean squared error (MSE) and R-squared. For two factors, the overall agreement rate and kappa are determined: `defaultSummary(ModelTest1)`

This example returns the values of the RMSE and metrics:

```
defaultSummary(ModelTest1)
```

```
##      RMSE  Rsquared      MAE
## 3.0098416 0.7978394 2.1373079
```

Let's try with another cross-validation method. In this case, we will adopt the k-fold technique that subdivides data into randomly selected subsets (named folds) of roughly equal sizes. One subset is used to validate the model, while the remaining subsets are used to train the model. This process is repeated k times, such that each subset is used exactly once for validation. The k-fold is a robust method for estimating accuracy, and the size of k and to tune the amount of bias in the estimate. k values are, in general, set to three, five, seven and ten. To perform the k-fold cross-validation technique, just use the `train()` function as follows:

```
Control1<-trainControl(method = "cv",number = 10)
```

```
LmFit2<-train(mpg ~ ., data = mtcars, method = "lm",trControl = Control1,metric="Rsquared")
```

The first line is used for setting the method adopted. To do this, the `trainControl()` function is used to control the computational methods of the `train()` function. In the `trainControl` function, the `cv` method is used (that means k-fold cross-validation), and the number of folds is set to 10. Then, the `train()` function is used to build a regression model with the k-fold cross-validation technique. Let's see a brief summary using the summary function:

```
summary(LmFit2)
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4506 -1.6044 -0.1196  1.2193  4.6271
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 12.30337    18.71788   0.657   0.5181
## cyl         -0.11144     1.04502  -0.107   0.9161
## disp         0.01334     0.01786   0.747   0.4635
## hp          -0.02148     0.02177  -0.987   0.3350
## drat         0.78711     1.63537   0.481   0.6353
## wt          -3.71530     1.89441  -1.961   0.0633 .
## qsec         0.82104     0.73084   1.123   0.2739
## vs           0.31776     2.10451   0.151   0.8814
```

```
## am          2.52023    2.05665    1.225    0.2340
## gear        0.65541    1.49326    0.439    0.6652
## carb       -0.19942    0.82875   -0.241    0.8122
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.65 on 21 degrees of freedom
## Multiple R-squared:  0.869, Adjusted R-squared:  0.8066
## F-statistic: 13.93 on 10 and 21 DF, p-value: 3.793e-07

PredictedTest2<-predict(LmFit2,mtcars)
ModelTest2<-data.frame(obs = mtcars$mpg, pred=PredictedTest2)
defaultSummary(ModelTest2)
```

```
##      RMSE Rsquared      MAE
## 2.1469050 0.8690158 1.7227402
```

Let's deal with another cross-validation method, LOOCV. This technique subdivides data using the k-fold approach, where k is equal to the total number of observations in the data. This time, a single observation is used for the testing set, and the remaining observations make up the training set.

The procedure is similar to that seen for the previous case. Again, we will use the `trainControl()` function to control the computable methods of the `train()` function, and then use the `train()` function to build a regression model:

```
Control2<-trainControl(method="LOOCV")
LmFit3<-train(mpg ~ ., data = mtcars, method = "lm", trControl = Control2)
summary(LmFit3)
```

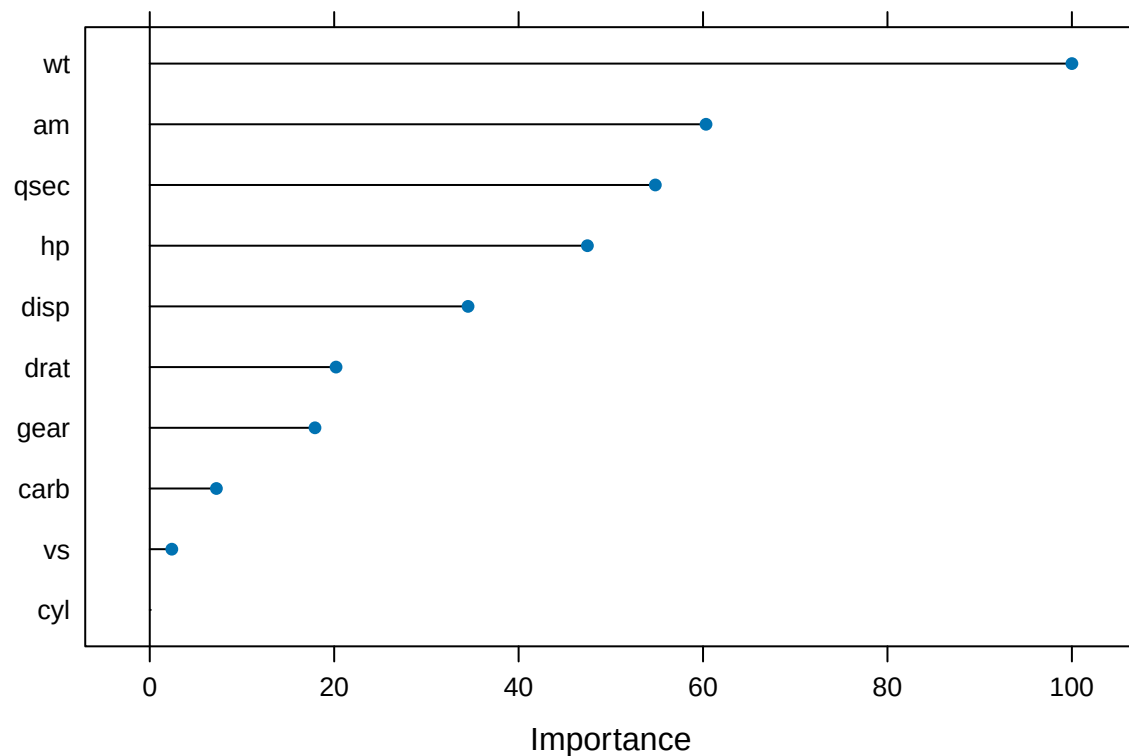
```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4506 -1.6044 -0.1196  1.2193  4.6271
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 12.30337    18.71788   0.657   0.5181
## cyl         -0.11144     1.04502  -0.107   0.9161
## disp         0.01334     0.01786   0.747   0.4635
## hp          -0.02148     0.02177  -0.987   0.3350
## drat         0.78711     1.63537   0.481   0.6353
## wt          -3.71530     1.89441  -1.961   0.0633 .
## qsec         0.82104     0.73084   1.123   0.2739
## vs           0.31776     2.10451   0.151   0.8814
## am           2.52023     2.05665   1.225   0.2340
## gear         0.65541     1.49326   0.439   0.6652
## carb        -0.19942     0.82875  -0.241   0.8122
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.65 on 21 degrees of freedom
## Multiple R-squared:  0.869, Adjusted R-squared:  0.8066
## F-statistic: 13.93 on 10 and 21 DF, p-value: 3.793e-07
```

In the caret package, a method for computation of variable importance for regression models is also available. This is the varImp() function that computes the variable's importance for objects produced by the train() function. So, we calculate the relative importance of variables for the last model:

```
varImp(LmFit3)
```

```
## lm variable importance
##
##      Overall
## wt    100.000
## am     60.325
## qsec   54.826
## hp     47.462
## disp   34.516
## drat   20.202
## gear   17.917
## carb    7.225
## vs      2.392
## cyl     0.000
```

```
plot(varImp(LmFit3))
```



Finally, the bootstrap resampling method is used. This technique creates bootstrap samples by randomly collecting observations from the original dataset with replacements against which to evaluate the model. Typically, a large number of resampling iterations are performed:

```
Control3<-trainControl(method="boot", number=100)
LmFit4<-train(mpg ~ ., data = mtcars, method = "lm", trControl = Control3)
summary(LmFit4)
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4506 -1.6044 -0.1196  1.2193  4.6271
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 12.30337    18.71788   0.657  0.5181
## cyl         -0.11144     1.04502  -0.107  0.9161
## disp         0.01334     0.01786   0.747  0.4635
## hp          -0.02148     0.02177  -0.987  0.3350
## drat         0.78711     1.63537   0.481  0.6353
## wt          -3.71530     1.89441  -1.961  0.0633 .
## qsec         0.82104     0.73084   1.123  0.2739
## vs           0.31776     2.10451   0.151  0.8814
## am           2.52023     2.05665   1.225  0.2340
## gear         0.65541     1.49326   0.439  0.6652
## carb        -0.19942     0.82875  -0.241  0.8122
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.65 on 21 degrees of freedom
## Multiple R-squared:  0.869, Adjusted R-squared:  0.8066
## F-statistic: 13.93 on 10 and 21 DF, p-value: 3.793e-07

PredictedTest4<-predict(LmFit4,mtcars)
ModelTest4<-data.frame(obs = mtcars$mpg, pred=PredictedTest4)
defaultSummary(ModelTest4)

##      RMSE Rsquared      MAE
## 2.1469050 0.8690158 1.7227402
```